

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:
Гордеев Артём
K3339

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

ВВЕДЕНИЕ

Данная работа посвящена проектированию микросервисной архитектуры для монолитного бэкенд-приложения “UnhappyJob” – платформы для поиска работы. В ходе работы выполняется декомпозиция монолита на самостоятельные микросервисы с соблюдением принципа database-per-service, проектируются схемы баз данных для каждого сервиса, описываются эндпоинты межсервисного взаимодействия в формате OpenAPI, а также формируется план миграции.

Задача

Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service. Спроектировать микросервисную архитектуру, отразить взаимосвязь между микросервисами, описать способы их взаимодействия друг с другом. Спроектировать разделение БД на обособленные БД. Спроектировать и описать эндпоинты межсервисного взаимодействия в формате спецификации OpenAPI. Описать варианты запросов и ответов, задокументировать возможные ошибки. Сформировать документ с конкретными требованиями и шагами для разделения монолита на микросервисы.

Ход работы

Анализ текущей монолитной архитектуры

Текущее приложение представляет собой монолитное REST API, написанное на Go с использованием фреймворка Gin и ORM GORM. Все данные хранятся в единой БД PostgreSQL. Монолит включает следующие доменные области:

- **Аутентификация и авторизация** – регистрация, вход, смена пароля, получение текущего пользователя;
- **Профили** – управление профилями соискателей и работодателей;
- **Компании** – создание и редактирование компаний;
- **Резюме** – CRUD-операции для резюме с вложенными навыками, образованием и опытом;
- **Вакансии** – CRUD-операции для вакансий с фильтрацией;
- **Отклики** – подача откликов, просмотр, изменение статуса.

Таблица 1 – Таблицы текущей монолитной БД

Таблица	Описание
users	Учётные записи пользователей
job_seekers	Профили соискателей
employers	Профили работодателей

Таблица	Описание
companies	Компании работодателей
resumes	Резюме соискателей
skills	Навыки, привязанные к резюме
educations	Образование, привязанное к резюме
experiences	Опыт работы, привязанный к резюме
industries	Справочник отраслей
jobs	Вакансии
job_applications	Отклики на вакансии

Принципы декомпозиции

При разделении монолита на микросервисы использованы следующие принципы:

- 1) **Single Responsibility Principle** – каждый микросервис отвечает за одну бизнес-область;
- 2) **Database-per-Service** – каждый микросервис владеет собственной базой данных, исключая прямой доступ к данным других сервисов;
- 3) **Loose Coupling** – микросервисы взаимодействуют только через чётко определённые API, минимизируя зависимости;
- 4) **High Cohesion** – сущности, тесно связанные по бизнес-логике, группируются в один микросервис.

Выделенные микросервисы

На основе анализа доменных областей монолита выделены четыре микросервиса и один компонент API Gateway:

Таблица 2 – Перечень микросервисов

Микросервис	Ответственность	Таблицы БД
Auth Service	Аутентификация, авторизация, управление учётными записями	users
Profile Service	Управление профилями соискателей и работодателей, компании	job_seekers, employers, companies
Resume Service	CRUD резюме с навыками, образованием и опытом	resumes, skills, educations, experiences
Job Service	CRUD вакансий, справочники, отклики на вакансии	jobs, industries, job_applications

Дополнительно вводится компонент **API Gateway**, который выступает единой точкой входа для клиентских приложений, выполняет маршрутизацию запросов к микросервисам, валидацию JWT-токенов и rate-limiting.

Диаграмма микросервисной архитектуры

На рисунке 1 представлена общая архитектура системы с указанием протоколов взаимодействия между микросервисами.

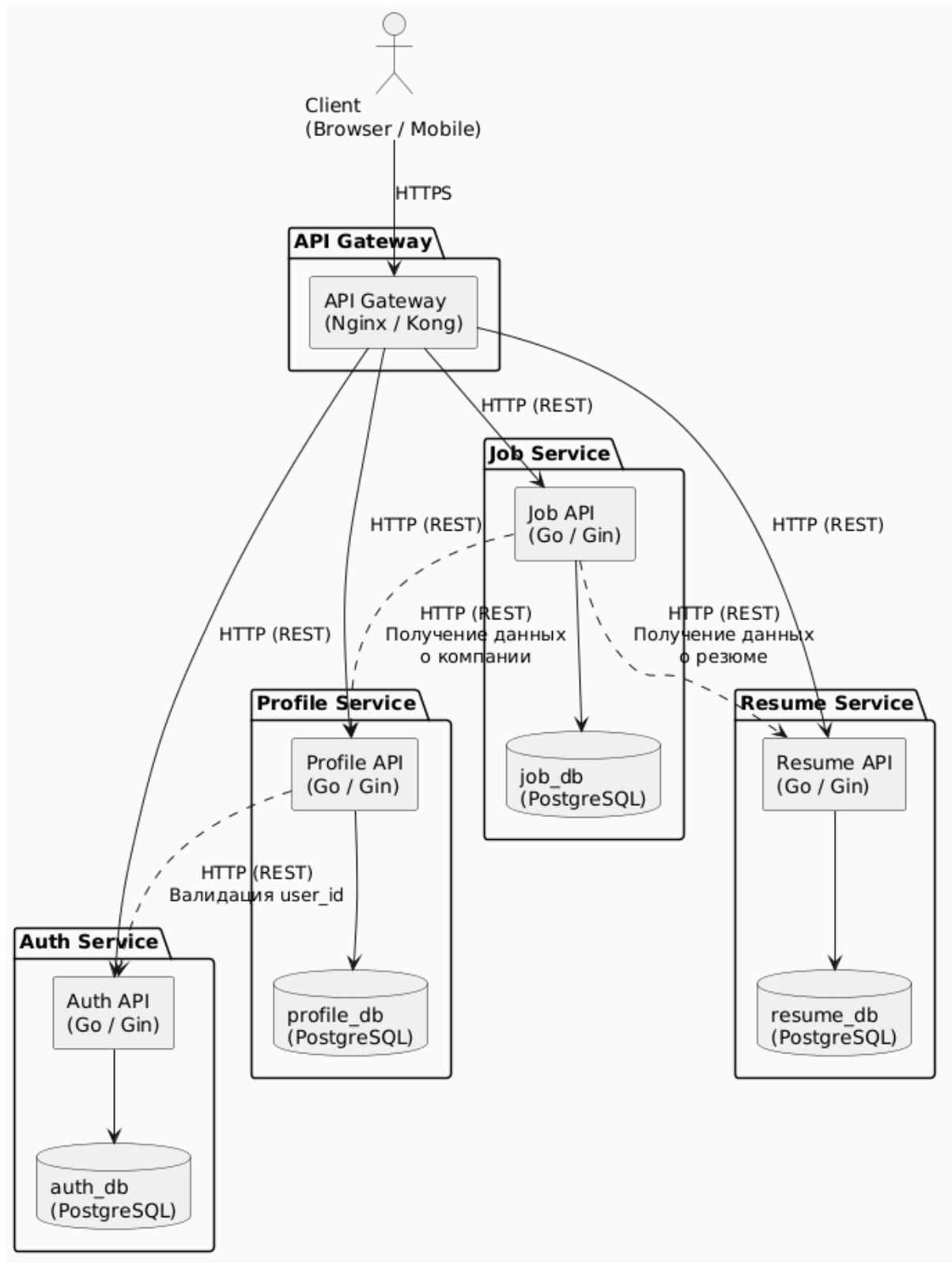


Рисунок 1 – Диаграмма микросервисной архитектуры

Способы взаимодействия между микросервисами

Для межсервисного взаимодействия используются два подхода:

Синхронное взаимодействие (REST over HTTP)

Используется для запросов, требующих немедленного ответа:

- **Job Service** → **Profile Service** – получение информации о компании при отображении вакансии;
- **Job Service** → **Resume Service** – получение данных резюме при просмотре откликов;
- **Profile Service** → **Auth Service** – валидация существования пользователя при создании профиля.

Передача контекста через JWT

API Gateway валидирует JWT-токен и прокидывает идентификатор пользователя (*X-User-ID*) и его роль (*X-User-Role*) в заголовках запроса к каждому микросервису. Это позволяет избежать повторной валидации токена в каждом сервисе.

Диаграмма последовательности: подача отклика на вакансию

На рисунке 2 представлена диаграмма последовательности для основного бизнес-процесса – подачи отклика на вакансию.

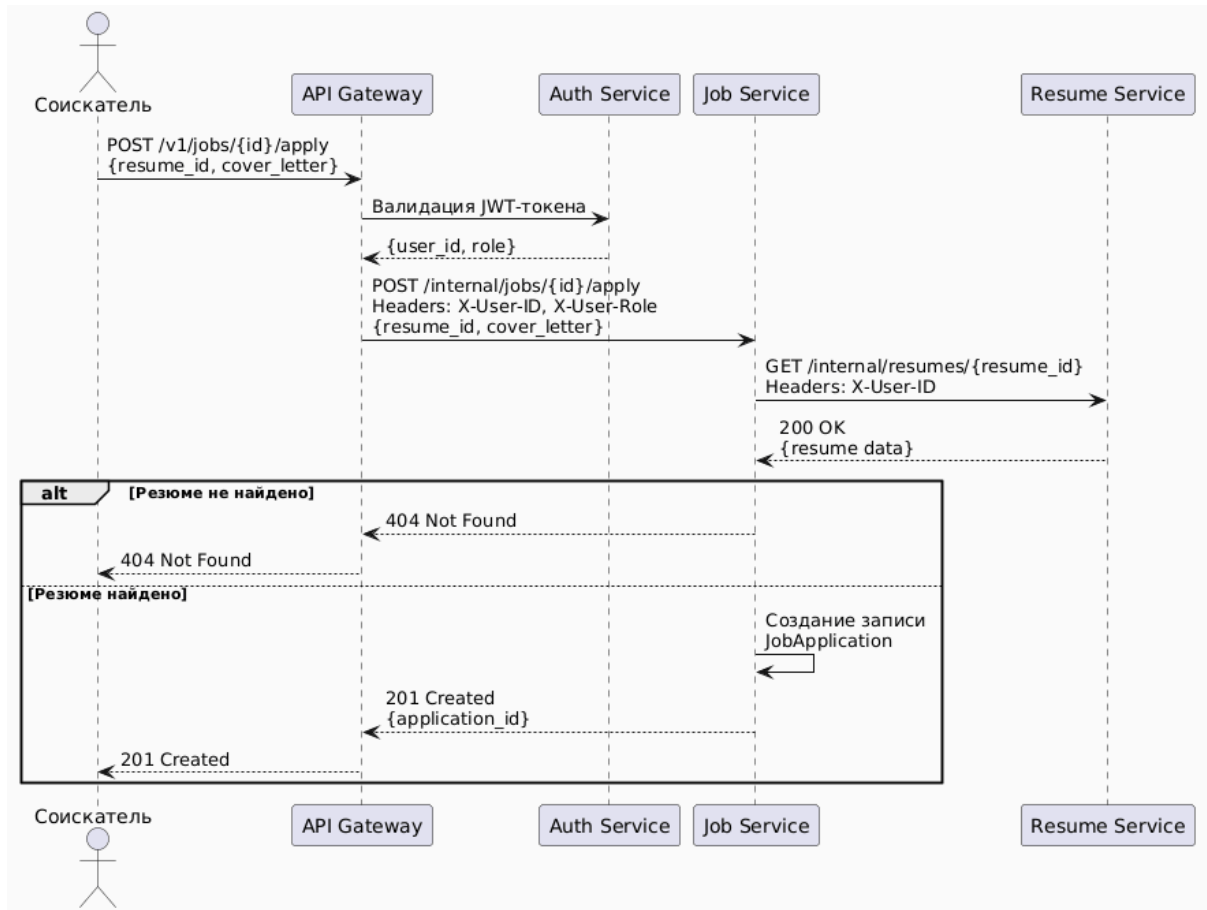


Рисунок 2 – Диаграмма последовательности – подача отклика

Разделение базы данных

Согласно принципу database-per-service, единая БД разделяется на четыре обособленные базы данных.

auth_db

Таблица 3 – Схема БД auth_db

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
role	VARCHAR(20)	NOT NULL, CHECK (role IN ('job_seeker', 'employer', 'admin'))
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

profile_db

Таблица 4 – Схема таблицы job_seekers (profile_db)

Столбец	Тип	Ограничения
user_id	UUID	PRIMARY KEY
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
phone	VARCHAR(30)	-

Таблица 5 – Схема таблицы employers (profile_db)

Столбец	Тип	Ограничения
user_id	UUID	PRIMARY KEY
company_id	UUID	NULLABLE, REFERENCES companies(id)
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
position	VARCHAR(100)	-

Таблица 6 – Схема таблицы companies (profile db)

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
name	VARCHAR(255)	NOT NULL
description	TEXT	-
website	VARCHAR(255)	-
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

resume_db

Таблица 7 – Схема таблицы resumes (resume db)

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
job_seeker_id	UUID	NOT NULL, INDEX
title	VARCHAR(255)	NOT NULL
summary	TEXT	-
salary_expectation	DECIMAL(12,2)	-
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

Таблица 8 – Схема таблицы skills (resume db)

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
resume_id	UUID	NOT NULL, REFERENCES resumes(id) ON DELETE CASCADE
name	VARCHAR(100)	NOT NULL

Таблица 9 – Схема таблицы educations (resume db)

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
resume_id	UUID	NOT NULL, REFERENCES resumes(id) ON DELETE CASCADE
institution	VARCHAR(255)	NOT NULL
degree	VARCHAR(255)	NOT NULL
start_date	TIMESTAMP	NOT NULL
end_date	TIMESTAMP	NULLABLE

Таблица 10 – Схема таблицы experiences (resume db)

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
resume_id	UUID	NOT NULL, REFERENCES resumes(id) ON DELETE CASCADE
company_name	VARCHAR(255)	NOT NULL
position	VARCHAR(255)	NOT NULL
description	TEXT	-
start_date	TIMESTAMP	NOT NULL
end_date	TIMESTAMP	NULLABLE

job_db

Таблица 11 – Схема таблицы industries (job db)

Столбец	Тип	Ограничения
id	SERIAL	PRIMARY KEY
name	VARCHAR(255)	NOT NULL, UNIQUE

Таблица 12 – Схема таблицы jobs (job db)

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
employer_id	UUID	NOT NULL, INDEX
company_id	UUID	NOT NULL, INDEX

Столбец	Тип	Ограничения
industry_id	INTEGER	NOT NULL, REFERENCES industries(id)
title	VARCHAR(255)	NOT NULL
description	TEXT	NOT NULL
requirements	TEXT	NOT NULL
salary_min	DECIMAL(12,2)	-
salary_max	DECIMAL(12,2)	-
currency	VARCHAR(3)	DEFAULT 'RUB'
experience	VARCHAR(30)	NOT NULL
status	VARCHAR(20)	NOT NULL, DEFAULT 'active'
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

Таблица 13 – Схема таблицы job applications (job db)

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
job_id	UUID	NOT NULL, REFERENCES jobs(id)
job_seeker_id	UUID	NOT NULL
resume_id	UUID	NOT NULL
cover_letter	TEXT	-
status	VARCHAR(20)	NOT NULL, DEFAULT 'sent'
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

Составной уникальный индекс: (*job_id, job_seeker_id*).

ER-диаграмма разделённых БД

На рисунке 3 представлена ER-диаграмма с указанием границ баз данных для каждого микросервиса.

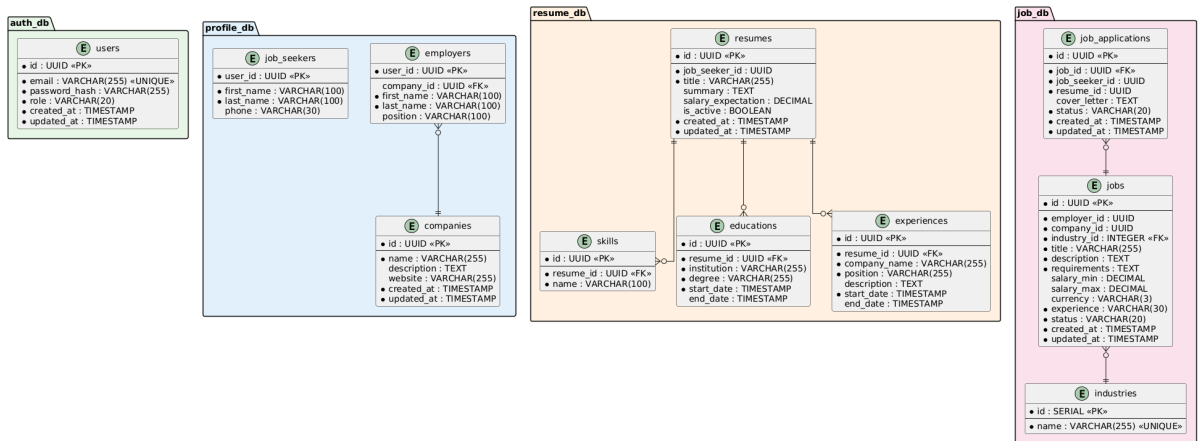


Рисунок 3 – ER-диаграмма разделённых баз данных

Спецификация эндпоинтов межсервисного взаимодействия (OpenAPI)

Ниже описаны внутренние (internal) эндпоинты, предназначенные исключительно для межсервисного взаимодействия. Эти эндпоинты недоступны извне и защищены сетевыми политиками.

Auth Service – внутренние эндпоинты

GET /internal/users/{user_id}

Получение данных пользователя по ID. Используется Profile Service для валидации существования пользователя.

Таблица 14 – GET /internal/users/{user_id} – параметры и ответы

Параметр	Тип	Расположение	Описание
user_id	UUID	path	Идентификатор пользователя

Ответ 200 OK:

Листинг 1 – Успешный ответ GET /internal/users/{user_id}

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "email": "user@example.com",
  "role": "job_seeker",
  "created_at": "2026-01-15T10:30:00Z"
}
```

Ответ 404 Not Found:

Листинг 2 – Ответ 404 GET /internal/users/{user_id}

```
{
  "error": "not_found",
  "message": "User not found"
}
```

POST /internal/users/batch

Получение данных нескольких пользователей по массиву ID. Используется для обогащения ответов при пакетных операциях.

Тело запроса:

Листинг 3 – Тело запроса POST /internal/users/batch

```
{
  "user_ids": [
    "550e8400-e29b-41d4-a716-446655440000",
    "550e8400-e29b-41d4-a716-446655440001"
  ]
}
```

Ответ 200 OK:

Листинг 4 – Успешный ответ POST /internal/users/batch

```
{
  "users": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "email": "user1@example.com",
      "role": "job_seeker"
    },
    {
      "id": "550e8400-e29b-41d4-a716-446655440001",
      "email": "user2@example.com",
      "role": "employer"
    }
  ]
}
```

Profile Service – внутренние эндпоинты

GET /internal/companies/{company_id}

Получение данных компании по ID. Используется Job Service для обогащения ответа при отображении вакансии.

Таблица 15 – GET /internal/companies/{company_id} – параметры и ответы

Параметр	Тип	Расположение	Описание
company_id	UUID	path	Идентификатор компании

Ответ 200 OK:

Листинг 5 – Успешный ответ GET /internal/companies/{company_id}

```
{
  "id": "660e8400-e29b-41d4-a716-446655440000",
  "name": "ООО Рога и Копыта",
  "description": "Крупная IT-компания",
  "website": "https://rogaikopyta.ru",
  "created_at": "2026-01-10T08:00:00Z"
}
```

Ответ 404 Not Found:

Листинг 6 – Ответ 404 GET /internal/companies/{company_id}

```
{
  "error": "not_found",
  "message": "Company not found"
}
```

GET /internal/employers/{user_id}

Получение данных профиля работодателя. Используется Job Service для проверки принадлежности работодателя к компании.

Таблица 16 – GET /internal/employers/{user_id} – параметры и ответы

Параметр	Тип	Расположение	Описание
user_id	UUID	path	ID пользователя (работодателя)

Ответ 200 OK:

Листинг 7 – Успешный ответ GET /internal/employers/{user_id}

```
{
  "user_id": "550e8400-e29b-41d4-a716-446655440001",
  "company_id": "660e8400-e29b-41d4-a716-446655440000",
}
```

```

    "first_name": "Иван",
    "last_name": "Петров",
    "position": "HR-менеджер"
}

```

Ответ 404 *Not Found*:

Листинг 8 – Ответ 404 GET /internal/employers/{user_id}

```

{
  "error": "not_found",
  "message": "Employer profile not found"
}

```

GET /internal/job-seekers/{user_id}

Получение данных профиля соискателя. Используется Job Service для обогащения данных при просмотре откликов.

Таблица 17 – GET /internal/job-seekers/{user_id} – параметры и ответы

Параметр	Тип	Расположение	Описание
user_id	UUID	path	ID пользователя (соискателя)

Ответ 200 *OK*:

Листинг 9 – Успешный ответ GET /internal/job-seekers/{user_id}

```

{
  "user_id": "550e8400-e29b-41d4-a716-446655440000",
  "first_name": "Алексей",
  "last_name": "Сидоров",
  "phone": "+79991234567"
}

```

Resume Service – внутренние эндпоинты

GET /internal/resumes/{resume_id}

Получение полных данных резюме по ID. Используется Job Service при подаче и просмотре откликов.

Таблица 18 – GET /internal/resumes/{resume_id} – параметры и ответы

Параметр	Тип	Расположение	Описание
resume_id	UUID	path	Идентификатор резюме

Таблица 19 – Заголовки запроса

Заголовок	Тип	Обязательный	Описание
X-User-ID	UUID	Нет	ID пользователя для проверки владения

Ответ 200 OK:

Листинг 10 – Успешный ответ GET /internal/resumes/{resume_id}

```
{
  "id": "770e8400-e29b-41d4-a716-446655440000",
  "job_seeker_id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "Go-разработчик",
  "summary": "3 года опыта в backend-разработке",
  "salary_expectation": 250000,
  "is_active": true,
  "skills": [
    { "id": "aaa...", "name": "Go" },
    { "id": "bbb...", "name": "PostgreSQL" }
  ],
  "educations": [
    {
      "id": "ccc...",
      "institution": "ИТМО",
      "degree": "Бакалавр",
      "start_date": "2022-09-01T00:00:00Z",
      "end_date": "2026-06-30T00:00:00Z"
    }
  ],
  "experiences": [
    {
      "id": "ddd...",
      "company_name": "Яндекс",
      "position": "Junior Go Developer",
      "description": "Разработка микросервисов",
      "start_date": "2025-01-15T00:00:00Z",
      "end_date": null
    }
  ]
}
```

```

    }
  ],
  "created_at": "2026-02-01T12:00:00Z"
}

```

Ответ 404 *Not Found*:

Листинг 11 – Ответ 404 GET /internal/resumes/{resume_id}

```

{
  "error": "not_found",
  "message": "Resume not found"
}

```

GET /internal/resumes/by-owner/{user_id}

Проверка принадлежности резюме конкретному пользователю. Используется Job Service для валидации при подаче отклика.

Таблица 20 – GET /internal/resumes/by-owner/{user_id} – параметры и ответы

Параметр	Тип	Расположение	Описание
user_id	UUID	path	ID соискателя
resume_id	UUID	query	ID резюме для проверки

Ответ 200 *OK*:

Листинг 12 – Успешный ответ GET /internal/resumes/by-owner/{user_id}

```

{
  "valid": true,
  "resume_id": "770e8400-e29b-41d4-a716-446655440000"
}

```

Ответ 404 *Not Found*:

Листинг 13 – Ответ 404 GET /internal/resumes/by-owner/{user_id}

```

{
  "valid": false,
  "error": "not_found",
  "message": "Resume not found or does not belong to user"
}

```

Job Service – внутренние эндпоинты

Job Service не предоставляет внутренних эндпоинтов для других микросервисов в текущей архитектуре, так как является конечным потребителем данных от Profile Service и Resume Service.

Полная спецификация OpenAPI для межсервисного взаимодействия

Листинг 14 – OpenAPI-спецификация межсервисного взаимодействия

```
openapi: 3.0.3
info:
  title: UnhappyJob Internal API
  description: >
    Внутренние эндпоинты для межсервисного взаимодействия
    платформы UnhappyJob.
  version: 1.0.0
servers:
  - url: http://auth-service:8081
    description: Auth Service
  - url: http://profile-service:8082
    description: Profile Service
  - url: http://resume-service:8083
    description: Resume Service

paths:
  # ===== AUTH SERVICE =====
  /internal/users/{user_id}:
    get:
      tags: [Auth Internal]
      summary: Получить данные пользователя по ID
      parameters:
        - name: user_id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        '200':
          description: Данные пользователя
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/InternalUserResponse'
        '404':
          $ref: '#/components/responses/NotFound'

  /internal/users/batch:
    post:
      tags: [Auth Internal]
      summary: Получить данные нескольких пользователей
      requestBody:
        required: true
        content:
          application/json:
            schema:
```

```

        type: object
        required: [user_ids]
        properties:
            user_ids:
                type: array
                items:
                    type: string
                    format: uuid
    responses:
        '200':
            description: Массив пользователей
            content:
                application/json:
                    schema:
                        type: object
                        properties:
                            users:
                                type: array
                                items:
                                    $ref:
'#/components/schemas/InternalUserResponse'

# ===== PROFILE SERVICE =====
/internal/companies/{company_id}:
    get:
        tags: [Profile Internal]
        summary: Получить данные компании по ID
        parameters:
            - name: company_id
              in: path
              required: true
              schema:
                  type: string
                  format: uuid
        responses:
            '200':
                description: Данные компании
                content:
                    application/json:
                        schema:
                            $ref:
'#/components/schemas/InternalCompanyResponse'
            '404':
                $ref: '#/components/responses/NotFound'

/internal/employers/{user_id}:
    get:
        tags: [Profile Internal]
        summary: Получить профиль работодателя по user_id
        parameters:
            - name: user_id
              in: path
              required: true

```

```

        schema:
            type: string
            format: uuid
    responses:
        '200':
            description: Профиль работодателя
            content:
                application/json:
                    schema:
                        $ref:
'#/components/schemas/InternalEmployerResponse'
        '404':
            $ref: '#/components/responses/NotFound'

/internal/job-seekers/{user_id}:
    get:
        tags: [Profile Internal]
        summary: Получить профиль соискателя по user_id
        parameters:
            - name: user_id
              in: path
              required: true
              schema:
                  type: string
                  format: uuid
        responses:
            '200':
                description: Профиль соискателя
                content:
                    application/json:
                        schema:
                            $ref:
'#/components/schemas/InternalJobSeekerResponse'
            '404':
                $ref: '#/components/responses/NotFound'

# ===== RESUME SERVICE =====
/internal/resumes/{resume_id}:
    get:
        tags: [Resume Internal]
        summary: Получить полные данные резюме
        parameters:
            - name: resume_id
              in: path
              required: true
              schema:
                  type: string
                  format: uuid
            - name: X-User-ID
              in: header
              required: false
              schema:
                  type: string

```

```

        format: uuid
        description: ID пользователя для проверки владения
responses:
  '200':
    description: Данные резюме
    content:
      application/json:
        schema:
          $ref:
'#/components/schemas/InternalResumeResponse'
  '404':
    $ref: '#/components/responses/NotFound'

/internal/resumes/by-owner/{user_id}:
get:
  tags: [Resume Internal]
  summary: Проверить принадлежность резюме пользователю
  parameters:
    - name: user_id
      in: path
      required: true
      schema:
        type: string
        format: uuid
    - name: resume_id
      in: query
      required: true
      schema:
        type: string
        format: uuid
  responses:
    '200':
      description: Результат проверки
      content:
        application/json:
          schema:
            type: object
            properties:
              valid:
                type: boolean
              resume_id:
                type: string
                format: uuid
    '404':
      description: Резюме не найдено или не принадлежит
      content:
        application/json:
          schema:
            type: object
            properties:
              valid:
                type: boolean

```

```

        example: false
        error:
          type: string
        message:
          type: string

components:
  schemas:
    InternalUserResponse:
      type: object
      properties:
        id:
          type: string
          format: uuid
        email:
          type: string
          format: email
        role:
          type: string
          enum: [job_seeker, employer, admin]
        created_at:
          type: string
          format: date-time

    InternalCompanyResponse:
      type: object
      properties:
        id:
          type: string
          format: uuid
        name:
          type: string
        description:
          type: string
        website:
          type: string
        created_at:
          type: string
          format: date-time

    InternalEmployerResponse:
      type: object
      properties:
        user_id:
          type: string
          format: uuid
        company_id:
          type: string
          format: uuid
          nullable: true
        first_name:
          type: string
        last_name:

```

```

        type: string
    position:
        type: string

InternalJobSeekerResponse:
    type: object
    properties:
        user_id:
            type: string
            format: uuid
        first_name:
            type: string
        last_name:
            type: string
        phone:
            type: string

InternalResumeResponse:
    type: object
    properties:
        id:
            type: string
            format: uuid
        job_seeker_id:
            type: string
            format: uuid
        title:
            type: string
        summary:
            type: string
        salary_expectation:
            type: number
        is_active:
            type: boolean
        skills:
            type: array
            items:
                type: object
                properties:
                    id:
                        type: string
                        format: uuid
                    name:
                        type: string
        educations:
            type: array
            items:
                type: object
                properties:
                    id:
                        type: string
                        format: uuid
                    institution:

```

```

        type: string
    degree:
        type: string
    start_date:
        type: string
        format: date-time
    end_date:
        type: string
        format: date-time
        nullable: true
    experiences:
        type: array
        items:
            type: object
            properties:
                id:
                    type: string
                    format: uuid
                company_name:
                    type: string
                position:
                    type: string
                description:
                    type: string
                start_date:
                    type: string
                    format: date-time
                end_date:
                    type: string
                    format: date-time
                    nullable: true
    created_at:
        type: string
        format: date-time

ErrorResponse:
    type: object
    properties:
        error:
            type: string
        message:
            type: string

responses:
    NotFound:
        description: Ресурс не найден
        content:
            application/json:
                schema:
                    $ref: '#/components/schemas/ErrorResponse'
    InternalError:
        description: Внутренняя ошибка сервиса
        content:

```

```
application/json:
  schema:
    $ref: '#/components/schemas/ErrorResponse'
```

Документирование возможных ошибок

Таблица 21 – Коды ошибок межсервисного взаимодействия

Код	Ошибка	Описание	Действие
200	-	Успешный запрос	-
400	bad_request	Некорректные параметры запроса	Проверить формат UUID и обязательные поля
404	not_found	Запрашиваемый ресурс не найден	Вернуть ошибку клиенту или использовать fallback
500	internal_error	Внутренняя ошибка сервиса	Повторить запрос с экспоненциальной задержкой
503	service_unavailable	Сервис временно недоступен	Использовать circuit breaker, вернуть 503 клиенту

Стратегия обработки ошибок

- 1) **Таймауты** – для всех межсервисных HTTP-запросов устанавливается таймаут в 3 секунды;
- 2) **Ретраи** – в случае получения 5xx-ошибки или таймаута выполняется до 3 повторных попыток с экспоненциальной задержкой (100ms, 200ms, 400ms);
- 3) **Circuit Breaker** – при превышении порога ошибок (50% за последние 10 запросов) цепь размыкается на 30 секунд;
- 4) **Graceful Degradation** – при недоступности Profile Service вакансии отдаются без данных о компании, с пометкой *company: null*.

Диаграмма компонентов

На рисунке 4 представлена диаграмма компонентов.

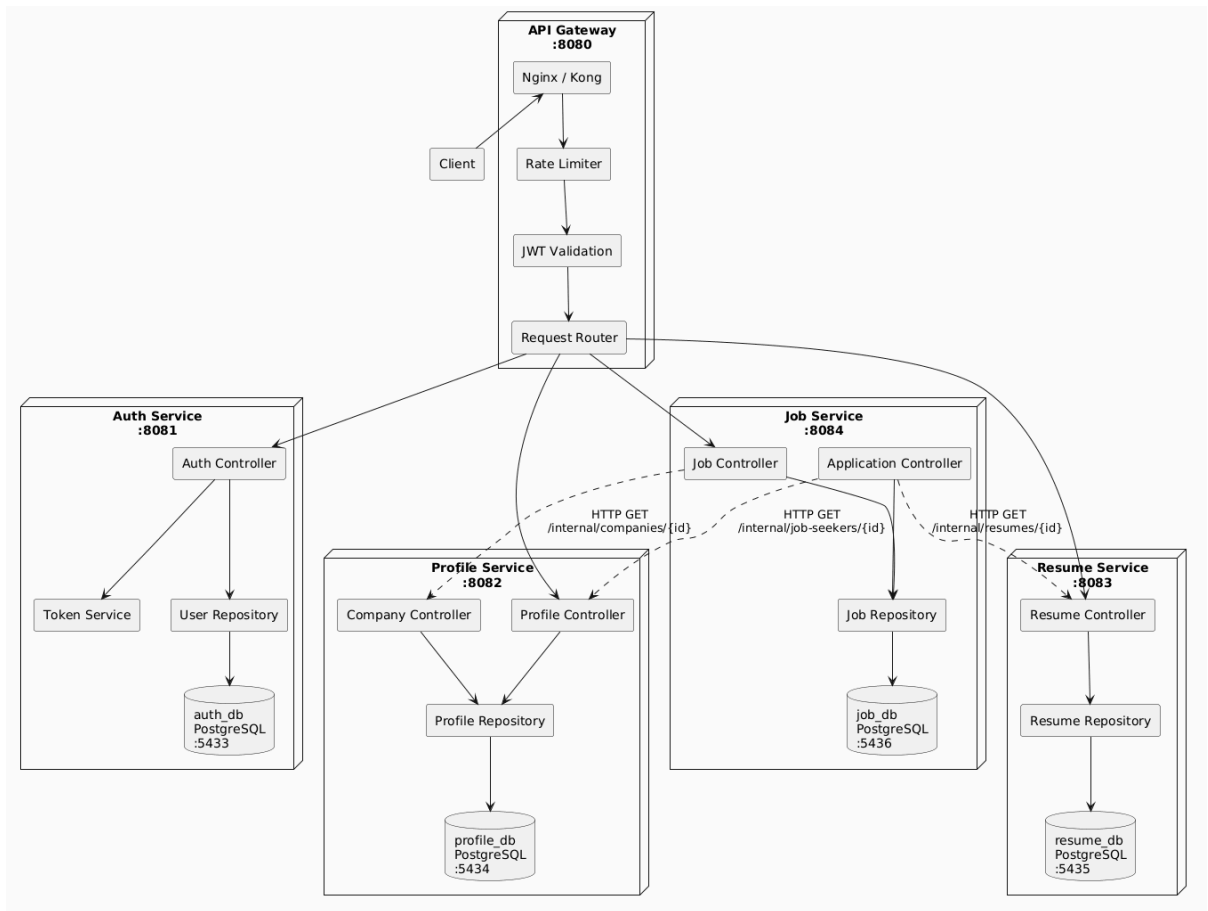


Рисунок 4 – Диаграмма компонентов системы

План миграции с монолита на микросервисы

Этап 1: Подготовка инфраструктуры

- 1) Настроить `docker-compose.yml` с четырьмя экземплярами PostgreSQL (`auth_db`, `profile_db`, `resume_db`, `job_db`);
- 2) Настроить API Gateway с маршрутизацией на основе URL-префиксов;
- 3) Настроить внутреннюю Docker-сеть для межсервисного взаимодействия.

Этап 2: Выделение Auth Service

- 1) Создать отдельный Go-модуль *auth-service*;
- 2) Перенести модель *User*, хендлеры аутентификации, middleware JWT;
- 3) Настроить подключение к *auth_db*;
- 4) Реализовать внутренний эндпоинт *GET /internal/users/{user_id}* и *POST /internal/users/batch*;
- 5) Настроить маршрутизацию */v1/auth/* в API Gateway.

Этап 3: Выделение Profile Service

- 1) Создать Go-модуль *profile-service*;
- 2) Перенести модели *JobSeeker*, *Employer*, *Company* и соответствующие хендлеры;
- 3) Настроить подключение к *profile_db*;
- 4) Реализовать внутренние эндпоинты для получения данных профилей и компаний;
- 5) Настроить маршрутизацию */v1/profile/*, */v1/companies/* в API Gateway.

Этап 4: Выделение Resume Service

- 1) Создать Go-модуль *resume-service*;
- 2) Перенести модели *Resume*, *Skill*, *Education*, *Experience* и хендлеры;
- 3) Настроить подключение к *resume_db*;
- 4) Реализовать внутренние эндпоинты для доступа к данным резюме;

5) Настроить маршрутизацию `/v1/resumes/` в API Gateway.

Этап 5: Выделение Job Service

- 1) Создать Go-модуль *job-service*;
- 2) Перенести модели *Job*, *Industry*, *JobApplication* и хендлеры;
- 3) Настроить подключение к *job_db*;
- 4) Реализовать HTTP-клиенты для обращения к Profile Service и Resume Service;
- 5) Настроить маршрутизацию `/v1/jobs/`, `/v1/industries`, `/v1/applications/` в API Gateway.

Этап 6: Миграция данных

- 1) Написать скрипты миграции для разделения данных из единой БД по четырём обособленным БД;
- 2) Выполнить миграцию в режиме downtime или blue-green deployment;
- 3) Верифицировать целостность данных после миграции.

Этап 7: Тестирование и запуск

- 1) Провести интеграционное тестирование всех межсервисных взаимодействий;

Маршрутизация API Gateway

Таблица 22 – Таблица маршрутизации API Gateway

URL-паттерн	Целевой сервис	Порт	Аутентификация
<code>/v1/auth/</code>	Auth Service	8081	Нет (кроме <code>/me</code> , <code>/logout</code> , <code>/change-password</code>)
<code>/v1/profile/</code>	Profile Service	8082	Да
<code>/v1/companies/</code>	Profile Service	8082	Частично (GET – нет)
<code>/v1/resumes/</code>	Resume Service	8083	Частично (GET <code>/:id</code> – нет)
<code>/v1/jobs/</code>	Job Service	8084	Частично (GET – нет)
<code>/v1/industries</code>	Job Service	8084	Нет

URL-паттерн	Целевой сервис	Порт	Аутентификация
/v1/applications/	Job Service	8084	Да

ЗАКЛЮЧЕНИЕ

В ходе выполнения домашнего задания была спроектирована микросервисная архитектура для монолитного бэкенд-приложения “UnhappyJob”. Монолит разделён на четыре микросервиса: Auth Service, Profile Service, Resume Service и Job Service, каждый из которых владеет собственной базой данных PostgreSQL в соответствии с принципом database-per-service.

Для каждого сервиса разработаны:

- схема обособленной базы данных;
- внутренние API-эндпоинты для межсервисного взаимодействия в формате OpenAPI;
- описание запросов и ответов с документированием возможных ошибок;
- стратегия обработки ошибок (таймауты, ретрай, circuit breaker).

Также разработан пошаговый план миграции, включающий подготовку инфраструктуры, последовательное выделение сервисов, миграцию данных и тестирование. Предложенная архитектура обеспечивает независимое масштабирование сервисов, изолированное развёртывание и повышенную отказоустойчивость системы.